

Consistency Testing of AI Compiler Stacks by Varying the Compilation

ANONYMOUS AUTHOR(S)

The same Python program can be compiled by `torch.compile` along several paths. Graph breaks are placed at positions that depend on the call site, automatic differentiation is traced only when a gradient is required, loops are vectorized only above a length threshold, execution may take place under a dispatch mode, autocast, or CUDA graphs, and a program may be exported and packaged for deployment. Existing testing techniques for AI compiler stacks vary the program and compile it along whichever path the harness takes, and they compare output values. Consistency failures between eager and compiled execution, however, often occur where a rarely used program feature meets a rarely taken path, and many of them do not change an output value. This paper varies the compilation instead and treats its path as a test dimension. We identify four path factors that a harness can enumerate and control one at a time: the position of graph breaks, the backend layer, the execution context, and the artifact route from export to a packaged runtime. Together with program-side factors derived from the source and from operator metadata, they form an execution matrix in which every difference from the reference execution has one candidate cause. Executions are compared on seven observables (values against a high-precision reference, metadata, object identity, exception type and routing, side effects, gradients, and process survival), and layered execution names the compiler stage at which a difference first appears. Applied to PyTorch 2.14 and nightly builds, the approach has produced 73 reports: 55 new issues, 15 issue comments, and 3 reports to a C++ compiler vendor. At the time of writing, 14 are fixed or have a fix under review, 10 are confirmed, 48 await triage, and 1 was rejected. On a 185-program corpus that program-only differential testing under three configurations had already covered, one path factor exposed two new defects; one program-side factor exposed 134 operators that compute under `torch.compile` on dtypes their eager kernels reject. Factor families for which no inconsistency was found are reported as well.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; **Compilers**; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: AI compiler stacks, `torch.compile`, consistency testing, varying the compilation, differential testing, metamorphic relation, graph breaks, JIT specialization

1 Introduction

Deep learning (DL) programs are written, reviewed, and debugged as Python code, but they are increasingly executed in another form. Since PyTorch 2, a call to `torch.compile` routes a model through bytecode-level graph capture, functionalization and automatic-differentiation tracing, operator decomposition, loop-level lowering, C++ or Triton code generation, guard-based specialization, and several caches [2]. Comparable pipelines exist in TVM [7], XLA-backed TensorFlow [1], MLIR-based stacks [18], and Triton [43]; we call such a pipeline an *AI compiler stack*. These systems are expected to produce an optimized artifact that is semantically equivalent to eager execution, so that the Python source remains the single description of what the program does.

This expectation can be violated without any visible sign in the source. An empirical study of *toxic compilation*¹ collected more than a thousand incidents in which the produced artifact diverged from the reviewed source. We call their common outcome a *source–artifact consistency failure*: for the same program, input, and context, eager and compiled execution differ in an observable way. Four of the defects reported in this paper illustrate the range. An `OrderedDict.move_to_end` call inside a compiled function is dropped, so that an LRU cache maintained next to the compiled step evicts the entry it has just used. An `IndexError` raised by a tensor operator bypasses the except clause written for it. A generator that leaves the compiled region is returned as an exhausted tuple

¹Reference withheld for double-anonymous review.

iterator whose return value is None. The expression $x * 1$ at the end of a graph returns the input tensor itself, so that an in-place update of the result modifies the input.

Compiler testing has so far varied the *program*. NNSmith [23], NeuRI [24], and HirGen [28] synthesize computation graphs; Tzer [25] and MLIRSmith [45] mutate intermediate representations; TorchProbe [40] applies semantics-preserving transformations to Python programs; FreeFuzz [48], DeepREL [11], and LLM-based fuzzers [9, 10] generate API invocations. In all of these techniques a program is compiled along one path, the one the harness takes by default. For `torch.compile` this leaves much of the compiler untested. Where TorchDynamo breaks the graph depends on the call site; whether AOTAutograd participates depends on whether a gradient is needed; whether TorchInductor emits a scalar or a vectorized loop depends on the tensor length; whether an operator takes a dedicated lowering or a generic decomposition depends on its dtype; a `TorchDispatchMode`, autocast, or CUDA graphs change which components run; export and ahead-of-time packaging replace the runtime. A defect on a path the harness does not take remains undetected regardless of how many programs are generated.

The compilation itself can, however, be varied: its path can be enumerated and controlled one factor at a time. The statements after which a graph break can be inserted, the backend layers that can be selected, the execution contexts that can be entered, and the artifact routes that can be taken are switches exposed by the pipeline; their number is small, their meaning is documented, and none of them is permitted to change the result of the program. The program-side factors that change compiler behavior can be enumerated as well: a static analysis of the source recovers the shapes, dtypes, scalars, and flags a compilation site reads, and operator metadata lists the dtypes and sample shapes each operator supports. Treating both groups as test dimensions, holding one fixed and changing the other by one factor at a time, yields tests in which every difference has a single candidate cause. A path factor is a semantics-preserving change to how the program is compiled, so the eager result is the expected outcome of a metamorphic relation [8]; the relation is the same for every factor, and the design effort goes into the choice of observables that are compared.

We implement this approach in Tcc. Section 3 defines the factor model, the one-factor execution matrix, an oracle over seven observables (values against a float64 or multi-precision reference, dtype and shape and stride, object identity and aliasing, exception type and its routing to a handler, side effects on inputs and external state, gradients, and process survival), and layered localization, which names the stage at which a difference first appears. Section 4 describes how the path factors are realized on `torch.compile` and how program-side factors are drawn from the source and from operator metadata.

Results. Reports filed against PyTorch 2.14 and nightly builds during the study number 73: 55 new issues, 15 comments that add members or root causes to existing issues, and 3 reports to the vendor of the C++ compiler that TorchInductor uses on Windows. At the time of writing, 14 are fixed or have a fix under review, 10 are confirmed by maintainers or reproduced by other contributors, 48 await triage, and 1 was judged expected behavior. A corpus of 185 programs that program-only differential testing under three configurations had already covered yielded two new Dynamo defects when one path factor, the position of graph breaks, was added. One metadata-derived factor exposed 134 operators that compute under `torch.compile` on dtypes their eager kernels reject. Of the 58 findings attributable to the compile pipeline, 39 are detectable only through an observable other than the output value. Guards, caches, compile composition, autocast, higher-order operators, custom operators, and the numerical behavior of the export path were consistent in every configuration we tried.

Contributions.

- Varying the compilation as a dimension of consistency testing, with a set of path factors that a harness can enumerate and control (graph-break position, backend layer, execution context, artifact route), and evidence that program-only testing does not reach the defects on rarely taken paths (Sections 3, 4, 5.2).
- A one-factor execution matrix over program-side and path-side factors and an oracle over seven observables, which make every difference attributable and make side-effect, identity, exception-routing, and process-level failures detectable (Sections 3, 5.4).
- An evaluation on injected faults, historical issues, and current PyTorch builds, with 73 reports classified by maintainer response and by root-cause location, and negative results that state where consistency holds (Section 5).

2 Background and Motivation

2.1 The torch.compile Stack

`torch.compile` wraps a Python callable and defers compilation to the first call [2]. Four layers then transform the program. *TorchDynamo* symbolically evaluates CPython bytecode, models Python objects with variable trackers, extracts an FX graph [35] of tensor operations, and installs *guards*: predicates over input metadata, Python values, and global state under which the captured graph may be reused. Code that Dynamo cannot model causes a graph break and falls back to the interpreter. *AOTAutograd* traces the graph with *fake tensors* that carry only metadata, removes mutation and aliasing through functionalization, and records a joint forward/backward graph. A *decomposition* table rewrites several hundred ATen operators into a smaller primitive set, and *meta kernels* compute output shapes, dtypes, and strides without data. *TorchInductor* lowers the primitives to a loop-level IR, fuses them, and generates C++/OpenMP code for CPUs or Triton kernels [43] for GPUs. Compiled code is kept in an in-memory cache keyed by guards and in on-disk caches keyed by graph and configuration hashes; the same graph can also be exported ahead of time and packaged by AOTInductor.

Each layer can be selected in isolation through the backend argument, which Tcc exploits for localization: `backend="eager"` executes the graph captured by Dynamo with eager kernels (E_1), `"aot_eager"` adds AOTAutograd and decompositions (E_2), and `"inductor"` adds lowering and code generation (E_3). Plain eager execution is E_0 .

2.2 Source–Artifact Consistency

Let f be a Python function, x an input, and c an execution context that fixes tensor metadata, Python-level values, global state, and compiler options. $E(f, x, c)$ denotes eager execution and $C(f, x, c)$ compiled execution. We compare executions through the observation

$$Obs(r) = \langle value, metadata, identity, exception, state, gradient, process \rangle,$$

where metadata covers shape, dtype, stride, device, and layout; identity records whether an output is an input object or shares its storage; exception covers the type of a raised exception and whether a handler in the program received it; state is the post-state of inputs, module buffers, and Python objects the program can reach; and process records whether the interpreter survived. The compiler is *consistent* on (f, x, c) if $Obs(E(f, x, c)) \approx Obs(C(f, x, c))$, where $\approx$ is equality except for a dtype-aware floating-point tolerance on values and gradients. For caches we additionally compare a cold compilation in context c_2 with a warm execution that first compiles under c_1 : $C_{cold}(f, x, c_2)$ versus $C_{warm}(f, x, c_1 \rightarrow c_2)$. If eager and cold agree but warm differs, the failure is attributed to specialization or cache reuse rather than to code generation. The context c has two parts: the program-side part (metadata, values, state) that the program can read, and the *compilation path*

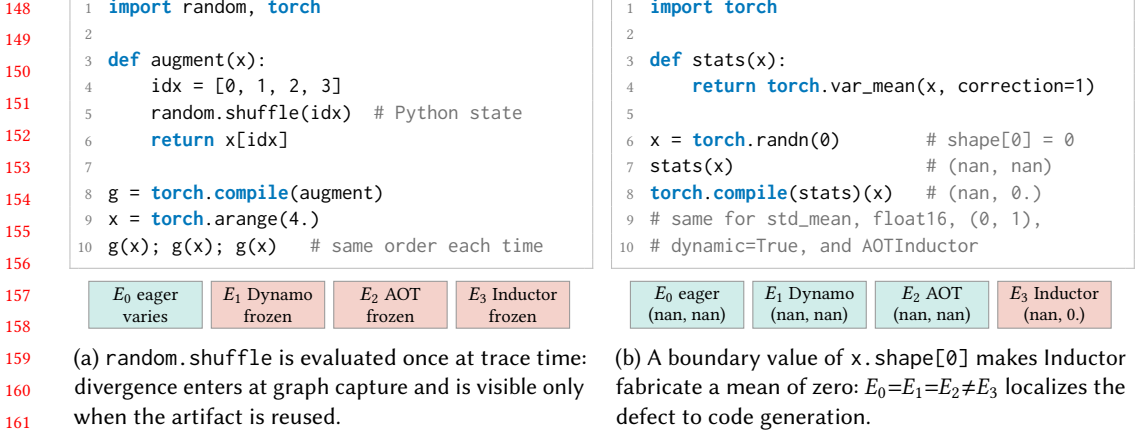


Fig. 1. Two source-artifact consistency failures found by Tcc in PyTorch 2.14. Each strip shows the outcome on eager execution and on the three compiled layers used for localization.

(graph-break positions, backend layer, execution context, artifact route) that the program cannot observe and that must therefore never change *Obs*. This paper varies both.

2.3 Motivating Examples

Figure 1 shows two defects found by Tcc in PyTorch 2.14 that illustrate why neither graph diversity nor an output-only oracle is sufficient.

A Python-semantics failure. In Figure 1a, eager execution advances the global random state, so consecutive calls return different permutations. Under torch.compile every call returns the permutation drawn during tracing. The outcome is identical for E₁, E₂, and E₃, which places the divergence in graph capture: Dynamo evaluates random.shuffle at trace time and embeds the result as a constant. No tensor operator is wrong, no exception is raised, and a single-call differential test passes; the defect is visible only when the same artifact is *reused*, which is the sequence dimension of our test model.

A code-generation failure on a boundary shape. In Figure 1b, torch.var_mean on an empty tensor returns (nan, nan) in eager mode and under E₁ and E₂, but (nan, 0.) under Inductor. The mean component is a fabricated zero. The trigger is a boundary value of the factor x.shape[0], and the layer pattern $E_0=E_1=E_2 \neq E_3$ assigns the defect to Inductor's reduction code. The operator is covered by PyTorch's own test suite, but not with this shape under the compiled path.

Observations. Both failures arise where a *legal but uncommon context* meets a compilation path, both are invisible to a generator that varies only graph topology, and they differ in layer, observable, and trigger, which calls for a systematic model of context factors instead of one specialized oracle.

3 The Tcc Approach

Figure 2 gives an overview. A compiled execution is determined by three things: the program, its input, and the path the compiler takes. Tcc derives *factors* for the first two from the source and from operator metadata, derives factors for the third from the structure of the pipeline, and executes a matrix in which each test differs from a reference execution in exactly one factor. A fixed oracle

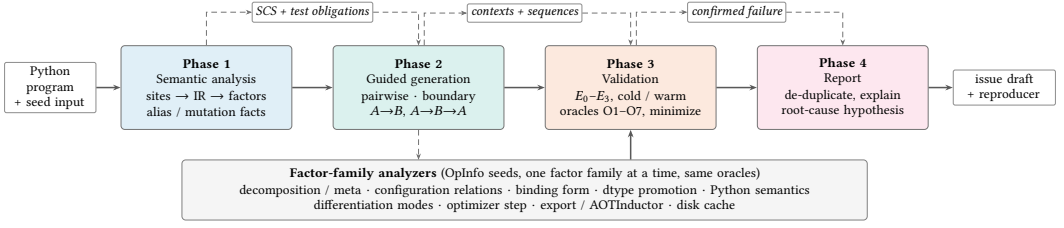


Fig. 2. Overview of Tcc. Phase 1 derives the context factors a compilation site reads; Phase 2 changes one factor at a time; Phase 3 compares eager and layered compiled executions, cold and warm; Phase 4 packages confirmed failures. The analyzers reuse Phases 3 and 4 for factor families that are not visible in user code.

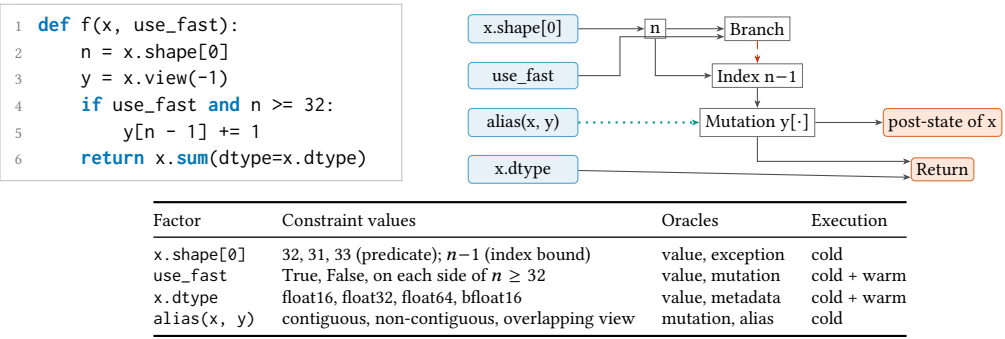


Fig. 3. Phase 1 on a small function. Solid edges are data dependencies, dashed edges control dependencies, dotted edges alias dependencies. Attributes that reach an observable (orange) become semantic factors; each factor yields a test obligation. Attributes the function never reads, such as $x.\text{stride}$, are pruned.

over seven observables decides whether the difference is a failure, and layered execution names the stage at which it appears.

3.1 Factor Model

Program-side factors from the source. Tcc locates *compilation sites*, the boundaries at which Python code enters the compiler (calls and decorators of `torch.compile` and project-specific wrappers), and lowers every function reachable from a site into a small dependency IR with data, control, alias, and state edges (Figure 3). A context attribute becomes a *semantic factor* of the site when a backward slice from a Return, Raise, Mutation, or StateWrite node reaches a read of that attribute: the rank, shape, stride, dtype, device, contiguity, and `requires_grad` of a tensor parameter; the value of a scalar, flag, string, or container; a global or closure variable. Comparisons and modulo tests on integer factors yield boundary values (for $x.\text{shape}[0] \geq 32$ the values 32, 31, 33); shape-index chains are recovered through constant offsets; writes through views yield alias and mutation facts that determine which inputs must be built as views or overlapping buffers. The factors of a site, with the backend and compile flags, form its *Semantic Context Signature* (SCS); attributes the program never reads are pruned. The analysis is syntactic and takes about 30 ms for our 29-program corpus.

Program-side factors from operator metadata. For operator-level testing the source is a single call, and the informative factors come from PyTorch’s OpInfo database, which lists for about 700 operators the supported dtypes, sample inputs, and reference implementations. Tcc derives three factor families from it: *boundary values*, which replace sample data by infinities, NaN, the dtype

extremes, values next to singularities (`nextafter(1, 0)` for inverse functions, $\pi/2$, $1 \pm \epsilon$) and magnitudes near overflow, in tensors long enough to reach vectorized code; *unsupported dtypes*, which cast a sample to a dtype the eager kernel rejects; and *dtype pairs* for binary operators. Each family is a change of one factor of one call; the operator and the shape stay fixed.

Path-side factors from the pipeline. The compilation path is determined by choices the pipeline exposes and documents as semantics-preserving. Tcc treats four of them as factors.

- *Graph-break position.* A call to `torch._dynamo.graph_break()` splits the captured graph and makes Dynamo generate a resume function that carries locals, side effects, exception state, and live objects across the break. Inserting the call after any statement must not change the result.
- *Backend layer.* `backend="eager"` executes the captured graph with eager kernels, `"aot_eager"` adds AOTAutograd and decompositions, `"inductor"` adds lowering and code generation. Each layer is a semantics-preserving refinement of the previous one.
- *Execution context.* A `TorchDispatchMode` that only forwards calls, `autocast`, `dynamic=True`, `fullgraph`, compile composition (a compiled function calling another, `compile(compile(f))`), and CUDA-graph or autotuning modes change which components run without changing what the program means.
- *Artifact route.* `torch.export`, its serialization round trip, AOTInductor packaging and loading, and the lifetime of the loaded runner replace the just-in-time runtime by a deployed one.

Every path factor has a reference execution in which the factor takes its default value, and the expected outcome is that the observables do not change. In the terminology of metamorphic testing [8, 37], each path factor is a semantics-preserving transformation of the compilation, the eager run is the source test case, and observable equality is the relation. Because the relation is the same for all factors, one oracle serves the whole matrix. Program-side factors are transformations of the input instead of the compilation, but they are tested against the same relation.

3.2 One-Factor Execution Matrix

Seeds. Tcc does not invent programs. A seed is a valid call of a compilation site, obtained from the function signature, from recorded calls, from reproducer scripts, from OpInfo samples, or from a corpus of small Python programs written for the features Dynamo models itself (the random module, exceptions and handlers, generators and their protocol, context managers, containers and their mutation, tensor subclasses; 274 programs at the time of writing). Programs in the corpus are added by mechanism: when a factor exposes a defect in one construct, programs that exercise the same mechanism in other constructs are written before the sweep is repeated (Section 5.2).

Single-factor variants. For each obligation derived from the SCS, Tcc builds contexts that differ from the seed in exactly one program-side factor, drawing values round-robin over obligations so that a small budget covers every factor once before any factor twice. For each path factor it builds the variant that changes only that factor: the corpus program with a graph break after every statement, the same call under another backend layer, inside a dispatch mode, or through the export route. A context in which eager execution of the mutated input raises is kept when the exception is the behavior under test; programs that eager execution rejects and the compiler accepts are a defect class of their own (Section 5.4).

Context sequences. Guards and caches are exercised by the execution matrix of Figure 4: for contexts A and B that differ in one factor, eager and cold-compiled runs of both, a warm run of B on the artifact compiled for A , and the return trip. The comparison of E_B with C_B^{cold} and with

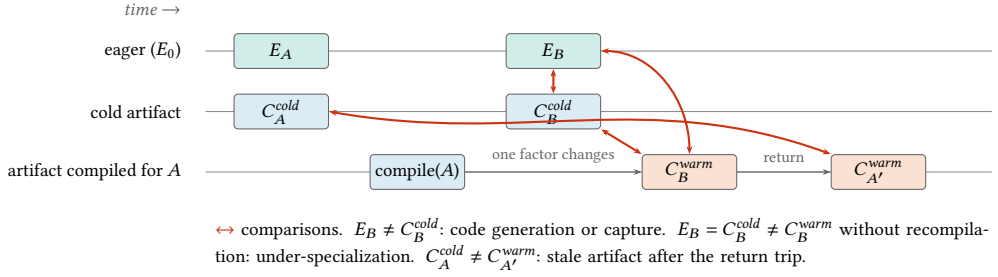


Fig. 4. Execution matrix for two contexts A and B that differ in one semantic factor. Recompilations are counted exactly, so each verdict is tied to an observed cache decision.

Table 1. The seven observables compared by Tcc and the mechanisms each one exposes. Cold and warm executions of Figure 4 are compared on every observable.

Observable	Compared observation	Typical mechanism exposed
O1 Value	scalars, tensors, nested containers; float64 or 30-digit reference with a dtype-aware noise floor; NaN/Inf pattern; sign of zero	wrong generated loop, unstable fused reduction, fabricated value on an empty reduction, vector polynomial with absolute-only accuracy
O2 Metadata	shape, dtype, stride, device, layout	wrong dtype promotion in a lowering, lost channels_last, ignored device=
O3 Identity	output is an input; storage sharing between inputs and outputs	no-op elimination that returns the input object, view replaced by a copy, generator replaced by an iterator
O4 Exception	raised or not; innermost exception type; whether the program's handler ran	missing validation, internal assertion, compile-only failure, exception that bypasses except
O5 State	post-state of inputs, module buffers, reachable Python objects; count of external effects	dropped or reordered in-place update, dropped container mutation, side effect replayed twice
O6 Gradient	input and parameter gradients against a higher-precision reference	wrong backward after tracing, lost in-place check, zero gradient from underflow
O7 Process	exit code and signal of the child process	native crash, hang, runtime destroyed while its threads run

C_B^{warm} separates code generation from under-specialization, and the return trip exposes incorrect invalidation. Recompilations are counted through Dynamo's frame counters with disk caches disabled.

3.3 Oracle

Table 1 lists the seven observables. Values and gradients are compared against a reference, not against a fixed tolerance: every test is also executed in eager mode with floating-point inputs promoted to float64, and for elementary functions against a 30-digit multi-precision library, and a compiled result is flagged only if its distance to the reference exceeds the distance of the eager result by a dtype-dependent factor, or if its NaN/Inf pattern or the sign of a zero differs. This rule accepts compiled results that are more accurate than eager execution and rejects one-ULP differences of re-associated reductions. Metadata covers dtype, shape, stride, device, and layout. Identity records whether an output is an input or shares its storage, because a compiler that returns the input object where eager execution returns a copy changes the meaning of every later in-place update. The exception observable compares the innermost exception of the `__cause__` chain and, when the program contains a handler, whether the handler ran. State covers the post-state of inputs, module

buffers, and Python objects reachable from the program (lists, dictionaries, generators), and the count of external effects such as printed lines. The process observable records exit codes and signals, since several findings terminate the interpreter.

3.4 Localization, Confirmation, and Reports

A failing test is re-executed on the backend layers E_1 – E_3 (and on the Triton path when a GPU is present). The first layer that disagrees with its predecessor names the suspected stage: graph capture, functionalization and decomposition, or code generation; a disagreement that appears only in a warm run names specialization or cache reuse; a disagreement that also appears without `torch.compile`, for example under a forwarding dispatch mode or in the eager kernel that the compiled path calls, is attributed *outside* the pipeline and reported as such. A candidate must survive a deterministic rerun with a fixed seed, a check that eager execution is deterministic on the input, and a rerun in a fresh interpreter; every work item of a sweep runs in a child process so that native crashes become findings instead of ending the sweep. Inputs are then minimized over rank, shape, values, container sizes, flags, and arguments, and the program is minimized by deleting statements outside the static slice of the triggering factor. A confirmed failure is emitted with the minimal reproducer, the observations on every layer, the factor that triggered it, and, where the generated code allows it, the responsible source location and a candidate patch. Likely duplicates are annotated, never filed; submission remains a human decision.

4 Realizing the Factors on `torch.compile`

Each factor family of Section 3.1 is realized as an *analyzer*: a sweep that takes seeds from one source, changes one factor, and reuses the oracle, confirmation, and reporting of Section 3.4. Table 2 lists them with the scale of one sweep on PyTorch 2.14. The path-side analyzers are new; the program-side ones extend the metadata-driven sweeps of earlier differential testers with boundary values and unsupported dtypes.

Graph-break position. The corpus programs are rewritten at the syntax-tree level so that `torch._dynamo.graph` follows every statement, either at the top level of the function or also inside loop, conditional, with, and try bodies; nested function definitions are left intact and closures are re-bound to the original cells so that the rewritten function sees the same state. The rewritten program is compared with eager execution and with the un-rewritten compiled program in the same process, so that the report lists the divergences the break *added*. Three variants replace the break by a statement with the same effect through a different mechanism: a call to a function wrapped in Dynamo’s `disable` decorator, a print, and a Python side effect on external state; the first two force a break with and without an observable effect, the third produces no break and tests side-effect replay alone.

Backend layer and execution context. The same seed is executed with backend set to each layer, with `dynamic=True`, under a `TorchDispatchMode` whose `__torch_dispatch__` only forwards the call (the mode is entered by the compile runtime itself, so a difference here means that the compiled program computes under a mode the user never wrote), under `autocast`, and under compositions of `torch.compile` that occur in application code (a compiled function calling another, `compile(compile(f))`, `fullgraph`, a warm-up in `no_grad` before a call with gradients, two backends on one function). For differentiable operators the sweep also compares input gradients. Every difference is confirmed without `torch.compile` where possible, which is how three findings were attributed to the dispatch machinery rather than to the compiler.

Artifact route. Each `OpInfo` sample is exported, serialized and deserialized, and executed; packaged with `AOTInductor`, loaded, and executed; and the loaded runner is destroyed while its outputs are

Table 2. Analyzers: the factor that is varied, the reference the result is compared with, and the scale of one sweep on PyTorch 2.14 (CPU unless noted). Path-side analyzers vary how the program is compiled; program-side analyzers vary the input or the operator call.

Side	Analyzer	Varied factor	Scale of one sweep
path	graph-break position	a break after every statement (top level or nested), by <code>graph_break</code> , <code>disable</code> , or <code>print</code> ; three backends; dynamic	274 programs, 660–816 insertion points, 6 min per backend
path	backend layer, context	eager / aot_eager / inductor; forwarding dispatch mode (forward and gradients); autocast; 17 compile compositions	955 differentiable op-dtype pairs; 530 ops $\times$ 2 autocast dtypes; 14 programs $\times$ 25 compositions
path	artifact route	<code>export</code> + <code>serialize</code> + <code>load</code> ; AOTInductor package + <code>load</code> ; runner destroyed after a run; CUDA-graph output reuse (19 patterns)	520 ops (CPU), 19 patterns (T4)
path	context sequences	one factor changed between two calls of one artifact; return trip; cross-process disk cache	26 switches, 29 cache cases
program	boundary values	inf, NaN, extremes, near-singular and near-overflow values in vector-length tensors; dynamic; multi-precision reference for 44 unary and 23 binary functions	1,203–1,345 op-dtype pairs per platform (CPU, T4)
program	unsupported dtypes	every dtype the eager kernel rejects	1,345 pairs (CPU), 1,303 (T4)
program	dtype pairs, scalars	8 $\times$ 8 dtype pairs in 4 argument forms; 11 binding forms of a scalar	17,664 cases; 386 ops
program	source factors (SCS)	one semantic factor of a compilation site at a time; boundary values from predicates	29 corpus programs, 114 obligations
program	Python corpus	274 programs of Dynamo-modeled Python features, two calls each	3 backends $\times$ 2 shape modes
program	decomposition, differentiation	Inductor decomposition executed eagerly; backward, complex, forward-mode, <code>vmap</code> ; 11 optimizers	576 ops, 11,911 variants; 327–678 ops per mode

alive and before the OpenMP worker threads of its last kernel have parked. Outputs are compared with eager execution and with `torch.compile` so that the report states whether a difference belongs to the route or to the shared lowering.

Program-side sweeps. The boundary-value analyzer replaces the data of every `OpInfo` sample by the edge values of Section 3.1 in tensors of at least the vector width, on CPU and CUDA; a variant compares elementary functions against a 30-digit reference near singularities and near overflow. The unsupported-dtype analyzer casts each sample to every dtype the eager kernel rejects and records whether the compiled program raises the same error, another, or none. The corpus analyzer runs the 274 Python programs twice each and compares return values, state, and exceptions with CPython; the differentiation analyzers compare backward, complex, forward-mode, and `vmap` traversals; the decomposition analyzer executes Inductor’s decompositions eagerly under a dispatch mode, without a C++ compiler in the loop.

Cost. Path-side analyzers reuse seeds, so their cost is low: a graph-break sweep over the corpus takes six minutes per backend, the dispatch-mode sweep covers 955 operator–dtype pairs in under an hour, and the artifact route covers 520 operators in 45 minutes. Resumable per-operator records make a native crash cost one operator, not the sweep.

5 Evaluation

The evaluation answers six research questions. **RQ1 (Varying the compilation):** what does varying the compilation find that varying the program does not? **RQ2 (Program dimension):** what do metadata-derived program-side factors find? **RQ3 (Oracle):** which findings require an observable other than the output value? **RQ4 (Attribution):** does layered execution name the stage that the developers eventually fix? **RQ5 (Comparison):** how does the one-factor matrix

compare with existing generators and with a cold-only comparison on a controlled benchmark?
RQ6 (Real-world defects): what did the approach find in current PyTorch builds, and how did the maintainers respond?

5.1 Experimental Setup

Implementation. Tcc is implemented in about 25,000 lines of Python on top of the standard ast module and PyTorch’s public and testing interfaces (backend=, Dynamo counters, OpInfo, TorchDispatchMode, fake tensors, torch.export, AOTInductor). Every work item runs in a child process with resumable per-item records. Disk caches are disabled unless they are the object of the test.

Subjects and environments. The subject is PyTorch 2.14.0, released during the study, and nightly builds of the development branch (2.15.0.dev, September 2026). Experiments ran on Windows 11 with MSVC and Python 3.14 (CPU), on Linux with GCC and Python 3.11/3.12 (CPU), and on Linux with a Tesla T4 GPU (CUDA 13, Triton). PyTorch 2.10.0 on the same Linux image serves as the older version for the historical benchmark.

Benchmarks and seeds. (i) A *controlled fault benchmark* of 19 faults injected through compiler backends that wrap the real pipeline and corrupt one decision (a generated value, a shape boundary or vectorization tail, a dtype, a branch, a dropped mutation or a view turned into a copy, an under-specialized guard, a stale or memoized cache entry), each attached to one of 29 corpus programs; the unmodified compiler on the same programs is the fixed version. (ii) A *historical benchmark* of 193 reproducers mined from the PyTorch issue tracker, run on PyTorch 2.10 and 2.14. (iii) *Seeds*: the 29-program corpus, the 274-program Python corpus, OpInfo samples for 703 operator entries, NNSmith-generated models, the mined reproducers, and 73 programs built from 14 torchvision architectures.

Baselines. Random context generation (B1) draws values for every factor family without consulting the program; NNSmith [23] (B3) is the generator closest to our setting that supports PyTorch 2. Both run inside our harness with the same budget, compiler, and machine, with and without our oracles. TorchProbe [40], FreeFuzz [48], and TitanFuzz [9] could not be executed against PyTorch 2.14 in our environment (Section 6.2).

5.2 RQ1: Varying the Compilation

One factor on a covered corpus. The Python corpus allows the program set, the inputs, and the oracle to be held fixed while only the path changes. Program-only differential testing had run the 185 programs of the corpus at that time under three configurations, backend="eager", dynamic=True, and aot_eager: 19 divergences, all of them known. Adding one path factor, a graph break after every statement, raised the count from 19 to 25 in the same process with the same oracle: eight new divergences (four programs, two calls each) and two that disappeared. The new ones belong to two defects, both silent: an OrderedDict.move_to_end on a dictionary that enters the compiled function is dropped by the side-effect replay (C41 in Table 4), and a generator that is alive across the break is reconstructed as an exhausted tuple iterator, so that .send() and .throw() raise AttributeError and StopIteration.value is None (C42). Both reproduce on Linux with Python 3.12 and on the nightly build. Neither needs the break once the mechanism is known: the first needs only a dictionary passed as an argument, the second only a generator that is returned. The break exposed them because it turns a local object created inside the frame into an input of the resume function. Replacing the break by a disable-wrapped call or by a print produced the same eight divergences; a pure Python side effect without a break produced none; breaks inside

loop, conditional, and try bodies (816 insertion points) added nothing to the top-level variant (660). Under the Inductor backend the same factor additionally exposed the identity family of C8 ($x * 1$ after a break returns the input object), which had been found earlier through a program-side factor.

Programs added after the findings. After C41 and C42, 57 programs were added. Most exercise the mechanisms involved (context managers, the generator protocol driven by the consumer, ordered containers passed as inputs, lazy built-ins with side effects); a smaller group covers exception flow around tensor work. Under the break factor this batch added eight divergences, all members of C41 and C42, and two on a `functools.lru_cache` wrapper whose inlining is documented behavior. On the unmodified path it exposed C44, an `IndexError` raised by a tensor operator that bypasses the program's own `except` clause, a mechanism unrelated to the break. The 33 programs added after C44 were written for that mechanism: they showed that every handler shape (except `Exception`, `contextlib.suppress`, a swallowing `__exit__`, nested calls, loops, generators) is bypassed and that `NotImplementedError` takes the same route (C45), while `TypeError`, `ZeroDivisionError`, assertions, and `except*` groups reach their handlers. Twenty-two of the 33 programs diverged, which delimits the scope of C44 without adding a defect.

Other path factors. The forwarding dispatch mode compared 955 differentiable operator–dtype pairs and their gradients; three operators differed (`linalg.pinv`, `matrix_sqrth`, `linalg.lstsq`, all complex), and confirmation without `torch.compile` attributed them to conjugate views that are not materialized under any dispatch mode (C32, C39), which also corrected the attribution of B1 from AOTAutograd to the dispatch machinery (C33). The artifact route compared 520 operators through export, serialization, and AOTInductor: the values agreed with `torch.compile` for every operator; the one route-specific finding is a process crash when the loaded runner is destroyed immediately after a run on Windows (C46). Compile composition, autocast, higher-order operators, custom operators, and CUDA-graph output reuse found nothing beyond documented behavior (Table 3).

Answer to RQ1. Of the 58 findings attributable to the compile pipeline, 20 came from a path-side factor. On a 185-program corpus that program-only testing under three configurations had already covered, one path factor exposed two new defects and one attribution correction; the 90 programs added afterwards found a third on the unmodified path and delimited it; the dispatch-mode and artifact-route factors each contributed a family that program-side variation does not reach.

5.3 RQ2: The Program Dimension

Metadata-derived factors keep the operator call fixed, so a sweep costs one compilation per pair. The unsupported-dtype analyzer ran 1,345 operator–dtype pairs on CPU: in 418 pairs of 134 operators the compiled program returned a value where the eager kernel raises `NotImplementedError` (C37), in five mechanisms (integer operators promoted to float, Boolean inputs, bitwise operators on floating-point bit patterns, half-precision special functions, complex normalizations); on the T4 the same factor gave 383 pairs, 73 of them CUDA-specific (C38). The boundary-value analyzer found nothing at the extreme values of the first version of its table and four functions after values next to singularities were added (`nextafter(1, 0)`, $1 \pm \epsilon$, $\sqrt{\max}$): `erfinv` loses four digits near ± 1 (C35), the vectorized float64 `acosh` overflows above 1.3×10^{154} (C36), and the vectorized float32 `erf` returns 0 for $|x| \leq 10^{-7}$ (C40). Comparing 44 unary functions against a 30-digit reference flagged six function–dtype pairs, of which `erf` and `atanh` were new; the same comparison for 23 binary functions and 18 reductions flagged nine, all of them families already reported (three by us); the analyzer thus re-finds known defects. Two lessons shaped the analyzer: probe tensors must be at least the vector width long, since a shorter probe tests only the scalar tail of the generated loop,

Table 3. Factor families that found no new inconsistency on PyTorch 2.14 (P = program side, Q = path side).

Side	Factor family	Scale	Outcome
Q	in-memory guards (dtype, layout, requires_grad, flags)	26 single-factor switches	always recompiled; cold = warm
Q	on-disk caches across processes	29 baked-in values and global switches	27 misses, 2 correct hits, 0 stale
Q	Inductor code-generation switches	593 ops $\times$ 20 configurations; 68 fused programs $\times$ 15	0 violations
Q	compile composition (compile(compile(f)), nested calls, disable, warm-up in no_grad, two backends)	14 programs $\times$ 25 compositions	21 differences, all documented behavior
Q	autocast, bfloat16 and float16	530 ops $\times$ 2 dtypes	0 dtype or value differences
Q	higher-order operators (cond, while_loop, scan, checkpoint)	47 programs $\times$ 3 backends	every difference is a raised error
Q	custom operators and autograd.Function	23 programs $\times$ 3 backends	0 silent (one dispatch-mode member of C32)
Q	export serialization and AOTInductor values	520 ops	values identical to torch.compile
Q	CUDA-graph output reuse (reduce-overhead)	19 access patterns (T4)	13 protected by the documented error, 6 unchanged, 0 stale
Q	graph breaks inside loop / conditional / try bodies; side-effect replay without a break	816 insertion points; 471 markers	nothing beyond the top-level variant; effects replayed once
P	vmap inside the compiled region; forward-mode AD with complex inputs	678 ops; 506 ops	0
P	binding form of scalar arguments	386 ops, 10,651 compilations	0 new
P	small real models (torchvision), CPU and GPU	131 programs	0
P	binary functions and reductions against a 30-digit reference	23 functions, 18 reductions	9 flagged, all previously reported

and eager execution is not a sufficient accuracy reference, since eager and compiled erf agree to 10^{-7} absolutely while the compiled result has no correct digit below 10^{-7} .

Answer to RQ2. Program-side factors derived from operator metadata account for 38 of the 58 pipeline findings; one factor, casting to a dtype the eager kernel rejects, exposed 134 operators, and a multi-precision reference exposed accuracy losses that an eager-versus-compiled comparison cannot see.

5.4 RQ3: Contribution of the Oracle

Table 4 names the observable that detected each issue. Of the 58 findings attributable to the pipeline, 19 are wrong values that a comparison of outputs against eager execution detects. The remaining 39 need another observable: 13 are failures of programs that eager execution accepts (internal assertions, C++ compilation errors, lowering exceptions), 9 are lost validations in which the compiled program returns where eager execution raises, 4 are dtype changes, 3 are identity changes (an output that is the input), 2 are wrong gradients with a correct forward value, 2 are dropped side effects, 2 are accuracy losses visible only against a multi-precision reference, 2 are process crashes, and 1 each are a memory-layout and a device change. The consequences are not local to the reported value: the identity change of C8 turns a later in-place update into corruption of the input, the dropped move_to_end of C41 makes an LRU cache evict the wrong entry, and the bypassed handler of C44 disables a fallback that works in eager mode.

Answer to RQ3. Thirty-nine of the 58 pipeline findings are invisible to an output-value comparison; exception parity in both directions accounts for 22 of them, and identity, state, gradient, precision, process, layout, and device observables for the rest.

5.5 RQ4: Attribution

On the controlled benchmark, the layer reported by localization matches the layer of the injected fault in all 19 cases. For real defects the ground truth is the fix. Thirteen PyTorch findings have a fix

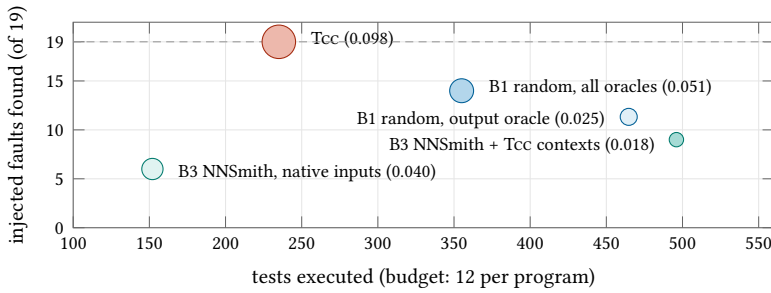


Fig. 5. RQ2: faults found against tests executed under the same harness, oracles, and budget. Marker area is proportional to test efficiency (unique failure clusters per executed test, in parentheses). B1 values are means over three seeds.

landed or under review; in all 13 the files changed by the fix lie in the layer that localization named (TorchDynamo for B2, B17, B26, C11; Inductor lowerings and decompositions for B8, B21, B23, B24; the reference layer for B14; AOTAutograd for C1; export packaging for B20; the shared eager kernel for B16). Ten findings were attributed outside the pipeline: three eager kernels reached through the compiled path (B16, C2, C4), three in torch.fx code generation and serialization (C9, C24, C27), and four in the dispatch machinery (B1 after correction, C32, C33, C39). They are reported and counted separately, because a differential oracle detects a divergence between two executions and cannot by itself decide which side is wrong.

Answer to RQ4. Localization named the layer of all 19 injected faults and the layer of all 13 fixed real defects; the same procedure attributed 10 findings to components outside the compile pipeline, which the paper counts apart.

5.6 RQ5: Comparison on the Controlled Benchmark

Figure 5 plots, for a budget of 12 tests per program, the injected faults found against the tests executed. Random generation with the output-only oracle that is customary in differential testing finds 11.3 of 19 faults (mean of three seeds) in 465 tests, 19% of which are invalid; giving it all seven observables raises it to 14, so one third of the gap is due to the oracle and two thirds to the choice of factors. NNSmith's models expose no Python-level factors, so even with our contexts added detection stays at 9. Tcc finds all 19 faults in 235 tests, with 16 invalid contexts, and raises no alarm on the unmodified compiler. An ablation that enables the analysis layers cumulatively (Figure 7) raises detection from 7 to 19 while the number of invalid tests falls from 109 to 16; alias facts and context sequences account for 9 of the 12 additional detections. Six faults require a warm artifact: a cold-only comparison finds none of them in 148 tests, random sequences find 5, and SCS-guided sequences find all 6 with 40% fewer tests (Figure 6). Against the real compiler the same sequences and 29 cross-process cache cases revealed no stale artifact. On the 193 mined reproducers (Figure 8), detection of open issues stays at 41% across PyTorch 2.10 and 2.14 while detection of closed issues drops from 41% to 13%, which yields 35 verified buggy/fixed pairs.

Answer to RQ5. Under identical oracles and budget, the one-factor matrix finds 19 faults against 14 for random generation and 9 for NNSmith with half the tests; on the real compiler, guards and caches were consistent, and the defects of RQ6 were reached through the paths behind them.

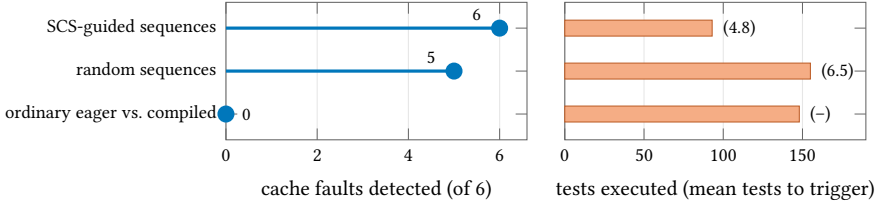


Fig. 6. RQ4: the six faults that need a warm artifact. A cold-only comparison cannot observe them; SCS guidance finds all six with fewer tests than random sequences.

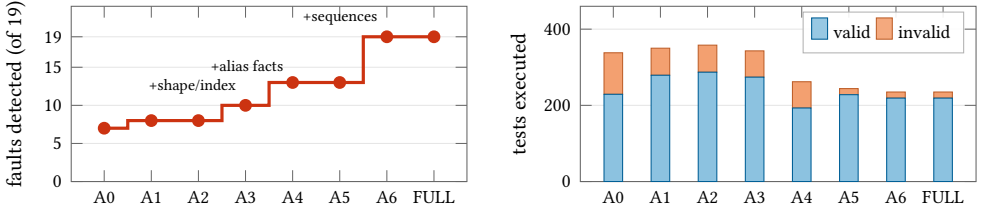


Fig. 7. RQ3: cumulative ablation of the analysis layers (A0: dynamic random contexts; A1: tensor metadata; A2: flow-sensitive dependencies; A3: shape-index relations; A4: alias and mutation facts; A5: SCS pruning; A6: context sequences). Detection rises while the number of tests, and in particular of invalid tests, falls.

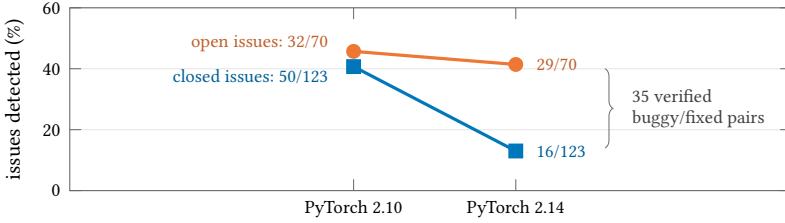


Fig. 8. RQ1 on 193 reproducers mined from the PyTorch issue tracker, executed on the same Linux CPU image with two PyTorch versions. Open issues remain detectable; closed issues stop being detected once their fix ships.

5.7 RQ6: Defects in Current PyTorch Builds

Reports. The sweeps on PyTorch 2.14 and nightly builds have produced 73 items to date (Table 4 lists the issues): 55 new issues, 15 comments that add members, platforms, or a corrected root cause to existing issues, and 3 reports to the vendor of the C++ compiler that TorchInductor uses on Windows, whose optimizer folds vector intrinsics that PyTorch's own vectorized kernels emit. Every candidate passed the confirmation steps of Section 3.4, was reproduced on PyTorch 2.14 and on a nightly build, on Windows and Linux (and on CUDA where applicable), and was searched in the issue tracker by root-cause keywords before filing. By maintainer response, 14 are fixed or have a fix under review, 10 are confirmed by a maintainer or reproduced by another contributor, 48 await a human response, and 1 was judged expected behavior (a mutated captured constant that export accepts and AOTInductor rejects). By root cause, 58 lie in the compile pipeline, 10 outside it, and 5 in other compilers (JAX, Numba, and the three MSVC reports). Of the 58, 42 are silent, 14 are failures of programs that eager execution accepts, and 2 terminate the process; they span

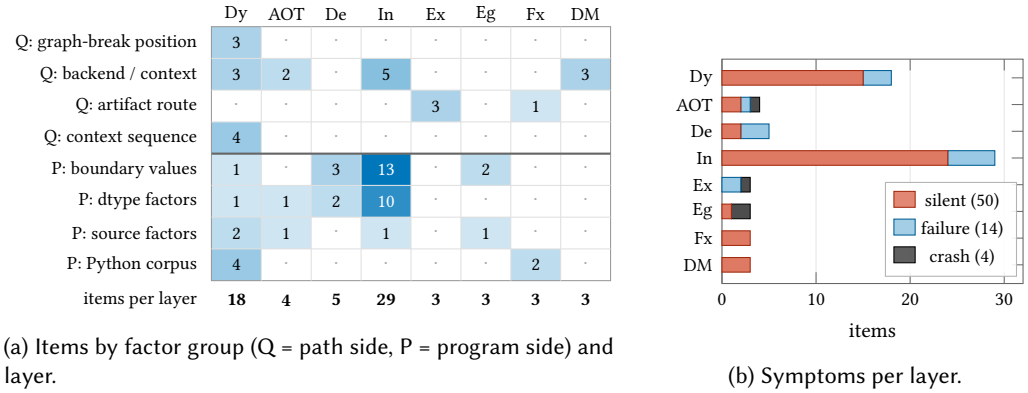


Fig. 9. RQ6: distribution of the 68 PyTorch items (issues and comments) by factor group, layer, and symptom. Layer abbreviations as in Table 4; the five cross-compiler items are omitted.

TorchDynamo (18), TorchInductor (29), AOTAutograd (4), decompositions and meta kernels (5), and export and AOTInductor (3). Every change to `torch.compile` passes a continuous-integration suite that already runs OpInfo-based compiler tests, and the tracker is triaged daily by dedicated teams.

Findings over time. The first 27 issues (B1–B27) were found with default paths; 14 of them are failures of programs that eager execution accepts. The issues found after the path factors and the metadata-derived factors were added (C1–C46) are silent in 26 of 28 cases and include the four Dynamo defects of RQ1, the identity defect C8, the 134-operator family C37, and the three accuracy defects. The maintainers’ own umbrella issue on lost validation, opened during the study, lists several of our reports as members.

Cost and negative results. The compile-based campaigns on default paths were the most expensive and the least productive: about 2,800 programs and 45,000 differential tests on default paths produced one new defect, the fabricated mean of Figure 1b. The path-side and metadata-derived analyzers produced the remaining defects at a fraction of the cost. Table 3 lists the factor families that found nothing new; they bound the claim of this paper and tell developers where consistency already holds.

Answer to RQ6. Tcc led to 73 reports against PyTorch 2.14 and nightly builds and two other compilers; 14 are fixed or have a fix under review and 10 are confirmed. Of the 58 pipeline findings 42 are silent, and none appears with a contiguous float32 tensor on the default path.

6 Discussion

6.1 Lessons

Varying the compilation. None of the 58 pipeline findings occurs with a contiguous float32 tensor on the default path, and 20 of them occur only when the path changes. RQ1 isolates this effect: with program set, inputs, and oracle held fixed on a corpus already run under three configurations, one additional path factor produced two defects within minutes. The explanation lies in the code that each factor reaches. A graph break makes every live local an input of a resume function, so the machinery that reconstructs Python objects and replays side effects is exercised for objects that a single-graph run never reconstructs. A dispatch mode routes composite operators through

a path on which conjugate views are materialized lazily. A packaged runtime has a lifetime that the just-in-time runtime does not have. Program diversity does not reach this code because the program cannot select the path.

Seeds written after a finding. Both corpus additions of RQ1 followed a finding. The first, written for the mechanisms behind C41 and C42, re-found those two under the break factor and exposed C44 through its exception-flow programs on the unmodified path; the second, written for C44, delimited it without adding a defect. A batch of 34 generic programs written before any finding produced nothing new.

Lost validation. Nine findings return a value where eager execution raises, and 13 raise where eager execution returns. The first group is not harmless: C37 computes convolutions on Boolean inputs and softmax on integers with results that no eager kernel defines, and the maintainers opened an umbrella issue on lost validation during the study. An oracle that ignores exceptions, or a harness that discards every context in which eager execution raises, misses this class. In the historical benchmark, fourteen open issues ran zero tests until the harness stopped discarding such seeds.

Reference strength. Three accuracy findings (C35, C36, C40) are invisible in an eager-versus-compiled comparison at ordinary inputs and appeared only against a multi-precision reference next to singularities. A float64 reference separates rounding noise from wrong results for well-conditioned computations but not for ill-conditioned ones: five closed historical issues that Tcc still flags had been judged intended behavior because a legal re-association is amplified by a later operator. Such findings are annotated for manual review; condition-aware suppression is future work.

6.2 Threats to Validity

Internal validity. Harness defects can appear as compiler defects. Two occurred during the study: a wrapper that compiled the wrong callee, and a sample generator whose no_grad finalizer ran inside a later trace and inserted a grad-mode node into exported graphs. Every candidate is therefore confirmed with a stand-alone script that does not import Tcc, in a fresh interpreter, on two PyTorch versions and two operating systems, and every report is re-verified from its text before filing. Injected faults may not represent real defects, so the evaluation also uses mined issues and defects found in current builds.

Construct validity. A differential oracle detects a divergence and does not decide which side is wrong; ten findings had their root cause outside the pipeline and are counted separately. Layer attribution was validated against the location of 13 fixes, which biases the check toward defects that were easy to fix. The maintainer-response counts reflect the tracker at the time of writing; most path-factor items were filed in the last days before submission. The comparison in RQ5 covers random generation and NNSmith; TorchProbe, the closest tool, could not be run against our subject.

External validity. The implementation targets torch.compile; the factor model needs only an eager reference, pipeline switches, and selectable layers, which other AI compiler stacks also offer but which we have not evaluated. Most experiments ran on CPUs; the T4 runs confirmed the platform-independent defects and found CUDA-specific members of C37, but Triton code generation is covered less densely than C++ code generation, and C46 is specific to Windows.

Table 4. The 55 issues filed against PyTorch and two other compilers (identifiers anonymized), in order of filing. *Layer*: Dy = TorchDynamo, AOT = AOTAutograd / autograd tracing, De = decompositions and meta kernels, In = TorchInductor, Ex = export / AOTInductor, Eg = eager kernel shared by the compiled path, Fx = torch.fx, DM = dispatch machinery, Ext = another compiler. *Sym.*: S = silent inconsistency, F = failure of a program that eager execution accepts, C = process crash. *Fac.*: P = program-side factor, Q = path-side factor. *Obs.*: observable that detected it (V value, D metadata, I identity, E exception, S state, G gradient, R precision reference, C process). *St.*: F fixed or fix under review, C confirmed, P pending, R rejected.

ID	Inconsistency (eager vs. compiled)	Layer	Sym.	Fac.	Obs.	St.
B1	linalg.pinv / matrix_sqrth: wrong gradient for complex inputs	AOT	S	P	G	P
B2	random.shuffle / sample return the same result on every call	Dy	S	Q	V	F
B3	jvp of ldxp terminates the interpreter under E_2	AOT	C	Q	C	P
B4	arg-reduction index emitted with Python ** in C++; compilation fails	In	F	Q	E	C
B5	new_zeros(device="cpu") returns a CUDA tensor	In	S	Q	D	C
B6	std / var: NaN for 10^{30} , zero value and zero gradient for 10^{-30}	In	S	P	V	P
B7	jvp of quantile / addr: internal assertion on every backend	Dy	F	Q	E	P
B8	ldxp result dtype wrong in both directions	In	S	P	D	F
B9	compiled autograd: interpolate backward hits an internal assertion	AOT	F	Q	E	P
B10	f-string format specifier on a symbolic size: internal Dynamo error	Dy	F	Q	E	C
B11	compiled SGD. step, foreach=True, complex parameters: lowering assertion	In	F	P	E	C
B12	interpolate on an empty spatial dim returns NaNs (out-of-bounds read); eager raises	De	S	P	E	C
B13	mutated captured tensor: export accepts, decomposition rejects, AOTInductor asserts	Ex	F	Q	E	R
B14	vector_norm(ord=inf) on an empty batch fails only for negative dim	De	F	P	E	F
B15	binary_cross_entropy / huber_loss: float32 result for half-precision inputs	In	S	P	D	P
B16	pdist backward on zero rows: integer division by zero kills the process	Eg	C	P	C	F
B17	random.seed inside the function is ignored on the first call	Dy	S	Q	V	F
B18	named tuple == with tensor fields: RecursionError (Python 3.14 only)	Dy	F	Q	E	P
B19	bmm under autotuning: missing symbol export in a C++ template (Windows)	In	F	Q	E	P
B20	AOTInductor cannot package models returning torch.return_types.*	Ex	F	Q	E	F
B21	lerp of Boolean tensors with a 0-d weight fails to compile	De	F	P	E	F
B22	sum / prod with dtype=torch.bool: C++ error / assertion	In	F	P	E	P
B23	addmm with a 0-d bias under autotuning: IndexError in lowering	In	F	Q	E	F
B24	var_mean / std_mean of an empty tensor: mean is 0 instead of NaN	In	S	P	V	F
B25	channel_shuffle loses the channels_last layout	In	S	P	D	C
B26	str(KeyError(k)) loses the quotes that CPython adds	Dy	S	P	V	F
B27	abs(complex, out=float64) rejected by the meta kernel	De	F	P	E	C
C1	in-place-modification check lost for a saved input view: silent wrong gradients	AOT	S	P	G	F
C2	adaptive_max_pool3d backward wrong for channels_last_3d (eager kernel)	Eg	S	P	G	C
C3	a changing Python float argument is baked into the cached graph	Dy	S	Q	V	P
C7	Numba: remainder of MIN_INT by -1 kills the process	Ext	C	-	C	P
C8	$x * 1 / x + 0$ as a graph output return the input object itself	In	S	P	I	C
C9	torch.fx codegen: operator.pow(-2, x) emitted as -2**x	Fx	S	P	V	F
C10	floating-point floor division off by one ($1.0 // 0.1$ gives 10)	In	S	P	V	C
C11	closure mutations in methods of a locally defined class are dropped	Dy	S	P	S	F
C12	JAX: gcd / lcm hang on the integer minimum	Ext	C	-	C	F
C13	compiled NumPy code: fix, cbrt, clip, sign return wrong values	Dy	S	P	V	P
C14	vectorized remainder: NaN for an infinite divisor, 0 for large quotients	In	S	P	V	P
C16	round(SymInt, -1) is the identity	Dy	S	P	V	P
C17	NaN self-comparison of an .item() value folded to a constant	Dy	S	P	V	P
C20	compiled NumPy code: dtype and shape rules differ (cumsum, square, median)	Dy	S	P	D	P
C22	clamp(int8, -1000, 1000) returns -24: bounds wrapped into int8	In	S	P	V	P
C24	torch.fx: parameters named nan / inf shadow float constants	Fx	S	P	V	P
C27	GraphModule pickle round trip turns in-place nodes into out-of-place ones	Fx	S	Q	S	P
C29	any(uint8) returns bool instead of uint8	In	S	P	D	P
C32	complex pinv / polar / matrix_sqrth wrong under any dispatch mode	DM	S	Q	V	P
C34	avg_pool backward with ceil_mode: full kernel size as divisor for the last window	In	S	P	G	P
C35	erfinv loses four digits near ± 1	In	S	P	R	P
C36	vectorized float64 acosh overflows above 1.3×10^{154}	In	S	P	V	P
C37	134 operators compute on dtypes the eager kernel rejects (418 pairs)	In	S	P	E	P
C40	vectorized float32 erf returns 0 below 10^{-7} ; atanh returns 0 below ϵ	In	S	P	R	P
C41	OrderedDict.move_to_end on an input dictionary is dropped	Dy	S	Q	S	P
C42	a generator that leaves the compiled region returns as an exhausted tuple iterator	Dy	S	Q	I	P
C44	IndexError from a tensor operator bypasses the program's except clause	Dy	S	P	E	P
C46	AOTInductor runner destroyed after a run: access violation (Windows, OpenMP workers still spinning)	Ex	C	Q	C	P

7 Related Work

Testing AI compiler stacks. Empirical studies show that DL compiler defects concentrate in high-level optimization and lowering and that a large share produce wrong results rather than crashes [22, 38]. Generators address this from the input side. NNSmith [23] and NeuRI [24] synthesize valid and diverse computation graphs from operator constraints, GenCoG [46] does so with a constraint DSL for TVM, HirGen [28] targets high-level IR, Tzer [25] mutates TVM's low-level IR together with pass sequences, and MLIRSmith [45] generates MLIR programs. WhiteFox [51] lets a language model read optimization source code to produce inputs that trigger it. Xiao et al. [49] apply metamorphic relations to compiled models. TorchProbe [40] is closest to our subject: it rewrites Python programs with semantics-preserving transformations that introduce dynamic language features and compares the variants under PyTorch 2. These techniques decide *which program* to compile, and each of them compiles it along one path. Tcc is orthogonal: for a given program it varies the compilation (graph-break position, backend layer, execution context, artifact route) and the program-side factors the program reads, one at a time, and compares identity, state, exception routing, gradients, metadata, and process survival besides values. TorchProbe's transformations and the cold/warm cache comparison of AlignGuard² are single members of this factor space; the configuration relations of [49] vary the model rather than the compiler. Equivalence modulo inputs [19] and metamorphic testing [8, 37] supply the form of our relation, observable equality under a semantics-preserving change; the change here is to the compilation rather than to the program.

Testing deep learning libraries. CRADLE [33], Audee [14], LEMON [47], and Muffin [12] compare models across frameworks or back ends. FreeFuzz [48] mines API invocations from documentation, tests, and models; DeepREL [11] infers relations between APIs; DocTer [50] and ACETest [39] extract input constraints from documentation and validation code; TitanFuzz [9] and FuzzGPT [10] generate API programs with language models; Predoo [55] studies operator precision; and ∇ Fuzz [52] differentially tests automatic differentiation. Studies of framework bugs [5, 13, 15] and of silent bugs [41] motivate oracles beyond crashes. Tcc borrows the seeds these communities created (OpInfo is PyTorch's counterpart to a mined API corpus) but asks a different question: not whether an API is correct, but whether its compiled execution agrees with its eager execution in every observable, including under forward-mode, compiled, and complex-valued differentiation, where our sweeps found defects (B1, B3, B7, B9) that eager-only AD testing cannot reach.

Compiler testing and validation. Differential testing [29], random program generation with Csmith [53] and YARPGen [26], and equivalence modulo inputs [19] are the foundations of compiler testing [6]; metamorphic testing [8, 37] and the oracle problem [3] frame what can be compared. For JIT compilers, Artemis [21] explores the space of compilation decisions and JIT-Picking [4] compares interpreter and JIT executions of JavaScript engines; our layered execution likewise uses the non-optimizing tier as a reference, with three intermediate tiers. Test-case reduction by delta debugging [54] and C-Reduce [36] inspires our minimizer, which additionally uses the static slice of the triggering factor. Verified compilation [20] and translation validation [27, 34] establish consistency by proof; they do not yet scale to an AI compiler stack with data-dependent guards, code generation through C++ and Triton, and caches, which is why a testing approach with strong oracles is the practical option today.

Source-artifact consistency and the software supply chain. Thompson's trusting-trust attack [42] is the classical example of an artifact that departs from its reviewed source. Reproducible builds [17],

²arXiv preprint, 2026.

in-toto [44], and Sigstore [30] secure the path from source to artifact against tampering, and taxonomies of supply-chain attacks [16, 31] catalog how that path is abused. These defenses establish *who* built an artifact and *from what*, not that a benign compiler preserved the semantics of benign source. Tcc addresses that gap for AI compiler stacks [2, 7, 32], where the artifact is generated, specialized, and cached at run time on the user’s machine.

8 Conclusion

A compiled deep learning program should behave like the Python source its developers reviewed on every path the compiler may take. Tcc tests this property by varying the compilation rather than only the program: graph-break positions, backend layers, execution contexts, and artifact routes are enumerated and changed one at a time, alongside program-side factors derived from the source and from operator metadata, and every execution is compared on seven observables. Reports against PyTorch 2.14 and nightly builds number 73 items, of which 14 are fixed or have a fix under review and 10 are confirmed. The path dimension accounts for 20 of the 58 pipeline findings and for both defects found on a corpus that program-only testing had already covered; observables beyond the output value account for 39. Guards, caches, compile composition, autocast, and the values of the export route were consistent. Future work will add condition-aware numerical oracles, denser coverage of the Triton path, and ports of the factor model to other AI compiler stacks.

Data Availability

The implementation of Tcc, the benchmarks, the sweep records, the classification of the 73 reports, and the stand-alone reproducer of every report are available in an anonymized artifact.

References

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Gordon Murray, Benoit Steiner, Paul A. Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-Scale Machine Learning.. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*. USENIX Association, 265–283. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- [2] Jason Ansel, Edward Z. Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation.. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024– 1 May 2024*. ACM, 929–947. doi:10.1145/3620665.3640366
- [3] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Trans. Software Eng.* 41, 5 (2015), 507–525. doi:10.1109/TSE.2014.2372785
- [4] Lukas Bernhard, Tobias Scharnowski, Moritz Schloegel, Tim Blazytko, and Thorsten Holz. 2022. JIT-Picking: Differential Fuzzing of JavaScript Engines.. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*. ACM, 351–364. doi:10.1145/3548606.3560624
- [5] Junjie Chen, Yihua Liang, Qingchao Shen, Jiajun Jiang, and Shuochuan Li. 2023. Toward Understanding Deep Learning Framework Bugs. *ACM Trans. Softw. Eng. Methodol.* 32, 6 (2023), 135:1–135:31. doi:10.1145/3587155
- [6] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2021. A Survey of Compiler Testing. *ACM Comput. Surv.* 53, 1 (2021), 4:1–4:36. doi:10.1145/3363562
- [7] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning.. In *13th USENIX Symposium on Operating Systems Design and Implementation*,

- OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018. USENIX Association, 578–594. <https://www.usenix.org/conference/osdi18/presentation/chen>
- [8] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Comput. Surv.* 51, 1 (2018), 4:1–4:27. doi:10.1145/3143561
- [9] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models.. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 423–435. doi:10.1145/3597926.3598067
- [10] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2024. Large Language Models are Edge-Case Generators: Crafting Unusual Programs for Fuzzing Deep Learning Libraries.. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 70:1–70:13. doi:10.1145/3597503.3623343
- [11] Yinlin Deng, Chenyuan Yang, Anjiang Wei, and Lingming Zhang. 2022. Fuzzing deep-learning libraries via automated relational API inference.. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*. ACM, 44–56. doi:10.1145/3540250.3549085
- [12] Jiazheng Gu, Xuchuan Luo, Yangfan Zhou, and Xin Wang. 2022. Muffin: Testing Deep Learning Libraries via Neural Architecture Fuzzing.. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 1418–1430. doi:10.1145/3510003.3510092
- [13] Qianyu Guo, Sen Chen, Xiaofei Xie, Lei Ma, Qiang Hu, Hongtao Liu, Yang Liu, Jianjun Zhao, and Xiaohong Li. 2019. An Empirical Study Towards Characterizing Deep Learning Development and Deployment Across Different Frameworks and Platforms.. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, San Diego, CA, USA, November 11-15, 2019*. IEEE, 810–822. doi:10.1109/ASE.2019.00080
- [14] Qianyu Guo, Xiaofei Xie, Yi Li, Xiaoyu Zhang, Yang Liu, Xiaohong Li, and Chao Shen. 2020. Audee: Automated Testing for Deep Learning Frameworks.. In *35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21-25, 2020*. IEEE, 486–498. doi:10.1145/3324884.3416571
- [15] Md Johirul Islam, Giang Nguyen, Rangeet Pan, and Hridesh Rajan. 2019. A comprehensive study on deep learning bug characteristics.. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*. ACM, 510–520. doi:10.1145/3338906.3338955
- [16] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains.. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 1509–1526. doi:10.1109/SP46215.2023.10179304
- [17] Chris Lamb and Stefano Zacchiroli. 2022. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Softw.* 39, 2 (2022), 62–70. doi:10.1109/MS.2021.3073045
- [18] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques A. Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation.. In *IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2021, Seoul, South Korea, February 27 - March 3, 2021*. IEEE, 2–14. doi:10.1109/CGO51591.2021.9370308
- [19] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs.. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14, Edinburgh, United Kingdom - June 09 - 11, 2014*. ACM, 216–226. doi:10.1145/2594291.2594334
- [20] Xavier Leroy. 2009. Formal verification of a realistic compiler. *Commun. ACM* 52, 7 (2009), 107–115. doi:10.1145/1538788.1538814
- [21] Cong Li, Yanyan Jiang, Chang Xu, and Zhendong Su. 2023. Validating JIT Compilers via Compilation Space Exploration.. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*. ACM, 66–79. doi:10.1145/3600006.3613140
- [22] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. The Deep Learning Compiler: A Comprehensive Survey. *IEEE Trans. Parallel Distributed Syst.* 32, 3 (2021), 708–727. doi:10.1109/TPDS.2020.3030548
- [23] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. 2023. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers.. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*. ACM, 530–543. doi:10.1145/3575693.3575707
- [24] Jiawei Liu, Jinjun Peng, Yuyao Wang, and Lingming Zhang. 2023. NeuRI: Diversifying DNN Generation via Inductive Rule Inference.. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on*

- the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023. ACM, 657–669. doi:10.1145/3611643.3616337
- [25] Jiawei Liu, Yuxiang Wei, Sen Yang, Yinlin Deng, and Lingming Zhang. 2022. Coverage-guided tensor compiler fuzzing with joint IR-pass mutation. *Proc. ACM Program. Lang.* 6, OOPSLA1 (2022), 1–26. doi:10.1145/3527317
- [26] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2020. Random testing for C and C++ compilers with YARPGen. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 196:1–196:25. doi:10.1145/3428264
- [27] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: bounded translation validation for LLVM. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*. ACM, 65–79. doi:10.1145/3453483.3454030
- [28] Haoyang Ma, Qingchao Shen, Yongqiang Tian, Junjie Chen, and Shing-Chi Cheung. 2023. Fuzzing Deep Learning Compilers with HirGen. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 248–260. doi:10.1145/3597926.3598053
- [29] William M. McKeeman. 1998. Differential Testing for Software. *Digit. Tech. J.* 10, 1 (1998), 100–107. <https://www.hpl.hp.com/hpjournal/dtj/vol10num1/vol10num1art9.pdf>
- [30] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. 2022. Sigstore: Software Signing for Everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*. ACM, 2353–2367. doi:10.1145/3548606.3560596
- [31] Marc Ohm, Henrik Plate, Arnold Sykosh, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment - 17th International Conference, DIMVA 2020, Lisbon, Portugal, June 24-26, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12223)*. Springer, 23–43. doi:10.1007/978-3-030-52683-2_2
- [32] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. 8024–8035. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- [33] Hung Viet Pham, Thibaud Lutellier, Weizhen Qi, and Lin Tan. 2019. CRADLE: cross-backend validation to detect and localize bugs in deep learning libraries. In *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019*. IEEE / ACM, 1027–1038. doi:10.1109/ICSE.2019.00107
- [34] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *Tools and Algorithms for Construction and Analysis of Systems, 4th International Conference, TACAS '98, Held as Part of the European Joint Conferences on the Theory and Practice of Software, ETAPS'98, Lisbon, Portugal, March 28 - April 4, 1998, Proceedings (Lecture Notes in Computer Science, Vol. 1384)*. Springer, 151–166. doi:10.1007/BFB0054170
- [35] James K. Reed, Zachary DeVito, Horace He, Ansley Ussery, and Jason Ansel. 2022. torch.fx: Practical Program Capture and Transformation for Deep Learning in Python. In *Proceedings of the Fifth Conference on Machine Learning and Systems, MLSys 2022, Santa Clara, CA, USA, August 29 - September 1, 2022*. mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2022/hash/7c98f9c7ab2df90911da23f9ce72ed6e-Abstract.html
- [36] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, Beijing, China - June 11 - 16, 2012*. ACM, 335–346. doi:10.1145/2254064.2254104
- [37] Sergio Segura, Gordon Fraser, Ana Belén Sánchez, and Antonio Ruiz Cortés. 2016. A Survey on Metamorphic Testing. *IEEE Trans. Software Eng.* 42, 9 (2016), 805–824. doi:10.1109/TSE.2016.2532875
- [38] Qingchao Shen, Haoyang Ma, Junjie Chen, Yongqiang Tian, Shing-Chi Cheung, and Xiang Chen. 2021. A comprehensive study of deep learning compiler bugs. In *ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23-28, 2021*. ACM, 968–980. doi:10.1145/3468264.3468591
- [39] Jingyi Shi, Yang Xiao, Yuekang Li, Yeting Li, Dongsong Yu, Chendong Yu, Hui Su, Yufeng Chen, and Wei Huo. 2023. ACETest: Automated Constraint Extraction for Testing Deep Learning Operators. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 690–702. doi:10.1145/3597926.3598088
- [40] Qidong Su, Chuqin Geng, Gennady Pekhimenko, and Xujie Si. 2023. TorchProbe: Fuzzing Dynamic Deep Learning Compilers. In *Programming Languages and Systems - 21st Asian Symposium, APLAS 2023, Taipei, Taiwan, November 26-29, 2023, Proceedings (Lecture Notes in Computer Science, Vol. 14405)*. Springer, 310–331. doi:10.1007/978-981-99-8311-7_15
- [41] Florian Tambon, Amin Nikanjam, Le An, Foutse Khomh, and Giuliano Antoniol. 2024. Silent bugs in deep learning frameworks: an empirical study of Keras and TensorFlow. *Empir. Softw. Eng.* 29, 1 (2024), 10. doi:10.1007/S10664-023-

- 10389-6
- [42] Ken Thompson. 1984. Reflections on Trusting Trust. *Commun. ACM* 27, 8 (1984), 761–763. doi:10.1145/358198.358210
- [43] Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations.. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL@PLDI 2019, Phoenix, AZ, USA, June 22, 2019*. ACM, 10–19. doi:10.1145/3315508.3329973
- [44] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. in-toto: Providing farm-to-table guarantees for bits and bytes.. In *28th USENIX Security Symposium, USENIX Security 2019, Santa Clara, CA, USA, August 14-16, 2019*. USENIX Association, 1393–1410. <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>
- [45] Haoyu Wang, Junjie Chen, Chuyue Xie, Shuang Liu, Zan Wang, Qingchao Shen, and Yingquan Zhao. 2023. MLIRSmith: Random Program Generation for Fuzzing MLIR Compiler Infrastructure.. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 1555–1566. doi:10.1109/ASE56229.2023.00120
- [46] Zihan Wang, Pengbo Nie, Xinyuan Miao, Yuting Chen, Chengcheng Wan, Lei Bu, and Jianjun Zhao. 2023. GenCoG: A DSL-Based Approach to Generating Computation Graphs for TVM Testing.. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 904–916. doi:10.1145/3597926.3598105
- [47] Zan Wang, Ming Yan, Junjie Chen, Shuang Liu, and Dongdi Zhang. 2020. Deep learning library testing via effective model generation.. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*. ACM, 788–799. doi:10.1145/3368089.3409761
- [48] Anjiang Wei, Yinlin Deng, Chenyuan Yang, and Lingming Zhang. 2022. Free Lunch for Testing: Fuzzing Deep-Learning Libraries from Open Source.. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 995–1007. doi:10.1145/3510003.3510041
- [49] Dongwei Xiao, Zhibo Liu, Yuanyuan Yuan, Qi Pang, and Shuai Wang. 2022. Metamorphic Testing of Deep Learning Compilers.. In *SIGMETRICS/PERFORMANCE '22: ACM SIGMETRICS/IFIP PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems, Mumbai, India, June 6 - 10, 2022*. ACM, 65–66. doi:10.1145/3489048.3522655
- [50] Danning Xie, Yitong Li, Mijung Kim, Hung Viet Pham, Lin Tan, Xiangyu Zhang, and Michael W. Godfrey. 2022. DocTer: documentation-guided fuzzing for testing deep learning API functions.. In *ISSTA '22: 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, South Korea, July 18 - 22, 2022*. ACM, 176–188. doi:10.1145/3533767.3534220
- [51] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2024. WhiteFox: White-Box Compiler Fuzzing Empowered by Large Language Models. *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024), 709–735. doi:10.1145/3689736
- [52] Chenyuan Yang, Yinlin Deng, Jiayi Yao, Yuxing Tu, Hanchi Li, and Lingming Zhang. 2023. Fuzzing Automatic Differentiation in Deep-Learning Libraries.. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1174–1186. doi:10.1109/ICSE48619.2023.00105
- [53] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers.. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2011, San Jose, CA, USA, June 4-8, 2011*. ACM, 283–294. doi:10.1145/1993316.1993532
- [54] Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Trans. Software Eng.* 28, 2 (2002), 183–200. doi:10.1109/32.988498
- [55] Xufan Zhang, Ning Sun, Chunrong Fang, Jiawei Liu, Jia Liu, Dong Chai, Jiang Wang, and Zhenyu Chen. 2021. Predoo: precision testing of deep learning operators.. In *ISSTA '21: 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, Denmark, July 11-17, 2021*. ACM, 400–412. doi:10.1145/3460319.3464843